华睿采集卡 SDK 开发手册

(C#篇)

Ver1.3

变更履历

编号	变更日期	版本号	变更内容概要	配套 SDK 版本
1	2022/11	Ver1.0	初版做成	2.4.0
2	2023/10	Ver1.1	增加消息通道接口	2.4.1
3	2023/04	Ver1.2	增加保存图片接口	2.4.2
4	2025/11	Ver1.3	增加 MVPImage 构造方式	2.6.1

目录

1. SDK 概要说明 4
1.1. SDK 支持的操作系统 4
1.2. SDK 安装目录 4
1.2.1. Windows 版 SDK 4
1.3. SDK（C#接口）对应的资源 5
1.4. SDK 开发工程配置 6
1.4.1. VS2010 ~ VS2022 6
2. SDK 的整体调用流程 9
3. SDK 的各功能接口调用说明 10
3.1. 系统操作 10
3.1.1. 枚举采集卡（发现设备） 10
3.1.2. 枚举相机设备（发现相机设备） 10
3.1.3. 获取版本信息 11
3.2. 采集卡操作 11
3.2.1. 采集卡的连接与关闭 11
3.2.2. 注册事件回调 11
3.3. 相机操作 12
3.3.1. 相机的连接与关闭 12
3.3.2. 注册事件回调 12
3.4. 相机操作 13
3.4.1. 流数据设置 13

3.4.2. 图像采集.....13
3.4.3. 获取图像数据.....14
3.4.4. 流与图像数据的其他操作.....15
3.5. 属性操作.....15
3.5.1. 采集卡与相机配置的下载、保存及加载配置.....15
3.5.2. 属性的判断、获取、设置.....16
3.5.3. 命令属性的执行.....21
3.5.4. 属性的查询与使用.....22
3.6. 图像操作.....29
3.6.1. 录像相关.....29
3.6.2. 图像相关.....30
4. 第三方平台的图像对象转换方法.....32
4.1. Halcon 对象（HObject）.....32
4.1.1. 相机图像格式为 Mono8 时.....32
4.1.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）...
4.2. Mvp 图像对象（MvpImage）.....33
4.2.1. 相机图像格式为 Mono8 时.....33
4.2.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）...
5. 配套例程说明.....34

1. SDK 概要说明

1.1. SDK 支持的操作系统

SDK 支持以下版本的操作系统

操作系统类型	操作系统版本号	备注
Windows	Win7 SP1 32/64bit	注意需要 SP1 补丁
	Win8 32/64bit	
	Win10 32/64bit	

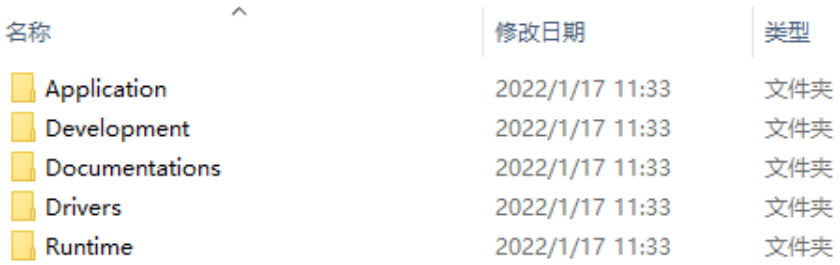
*1. 从 SDK2.2.5 开始已经不支持 windows xp 系统

1.2. SDK 安装目录

1.2.1. Windows 版 SDK

Windows 版本 SDK 的安装目录结构如下：

*1. 从 Ver2.1.0 版本开始，支持用户自己选择安装路径。



文件夹说明如下:

文件夹	描述
Application	1. 相机客户端软件，包含 32 位和 64 位 2. 相机配套工具软件（CamTools），包含 32 位和 64 位
Development	存放了 SDK 二次开发所需的各种资源文件 1. SDK 头文件、Lib 库 2. dotNet SDK 库 3. 第三方算法平台（Halcon、Sherlock）插件库 4. SDK 例程(C、MFC、C#、VB.Net、QT、Delphi、Python)
Documentations	.Net 接口文档 ActiveX 接口文档 C 接口文档 SDK 开发手册
Drivers	1. GigE 驱动安装、卸载程序 2. U3V 驱动安装、卸载程序 3. 虚拟串口驱动安装、卸载程序 4. DShowFilter 安装、卸载程序 5. PCIE 驱动安装、卸载程序
Runtime	SDK 运行时库（包含 GenICam V3.0 第三方库） 包含 32 位和 64 位

1.3. SDK（C#接口）对应的资源

大类别	小类别	文件位置	文件名
SDK 库	DOTNET_3.5（32 位）	【SDK 安装目录】\Development\DOTNET DLL\IMVFG\DOTNET_3.5\x86	MVFGSDK_Net.dll
	DOTNET_3.5（64 位）	【SDK 安装目录】\Development\DOTNET DLL\IMVFG\DOTNET_3.5\x64	MVFGSDK_Net.dll
	DOTNET_4.0（32 位）	【SDK 安装目录】\Development\DOTNET DLL\IMVFG\DOTNET_4.0\x86	MVFGSDK_Net.dll
	DOTNET_4.0（64 位）	【SDK 安装目录】\Development\DOTNET DLL\IMVFG\DOTNET_4.0\x64	MVFGSDK_Net.dll

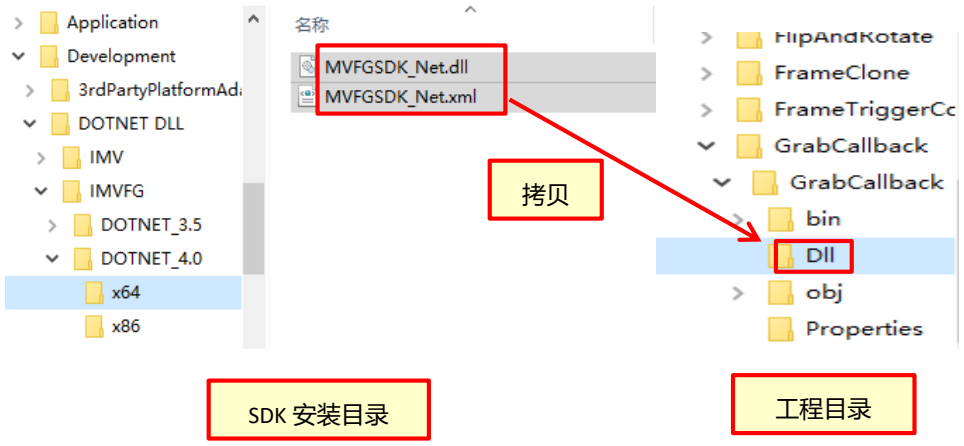
开发文档	中文	【SDK 安装目录】\Documentations\	IMVFG_API_Net_CHS.chm 采集卡 SDK 开发手册(C#篇).pdf
	英文	【SDK 安装目录】\Documentations\	IMVFG_API_Net_ENG.chm
开发例程	\	【SDK 安装目录】\Development \Samples \C#\IMVFG	

1.4. SDK 开发工程配置

1.4.1. VS2010 ~ VS2022

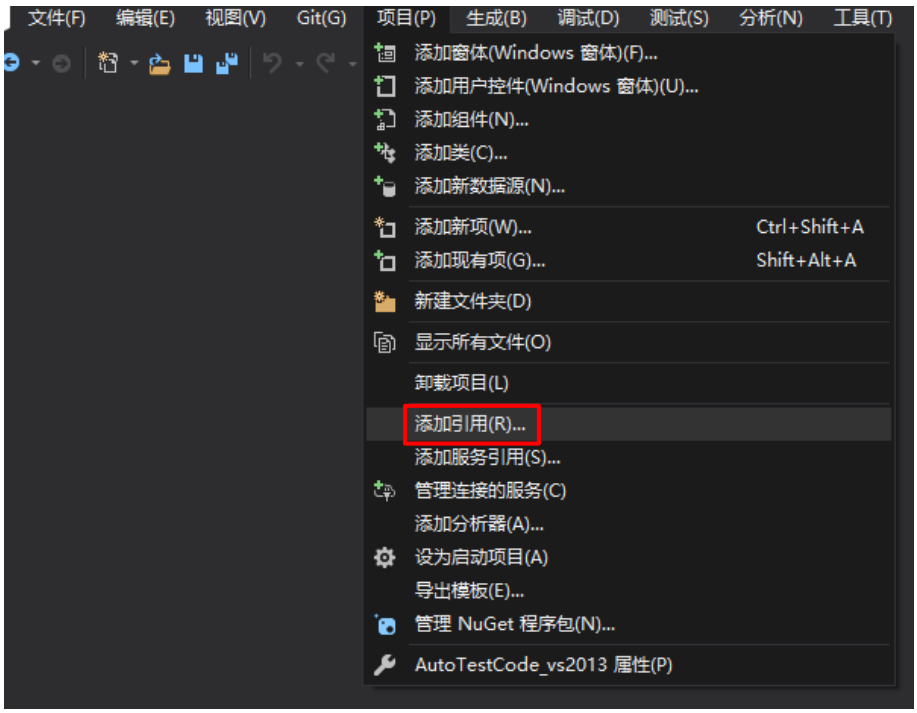
以 DOTNET4.0 64 位工程配置为例，说明配置工程方法。

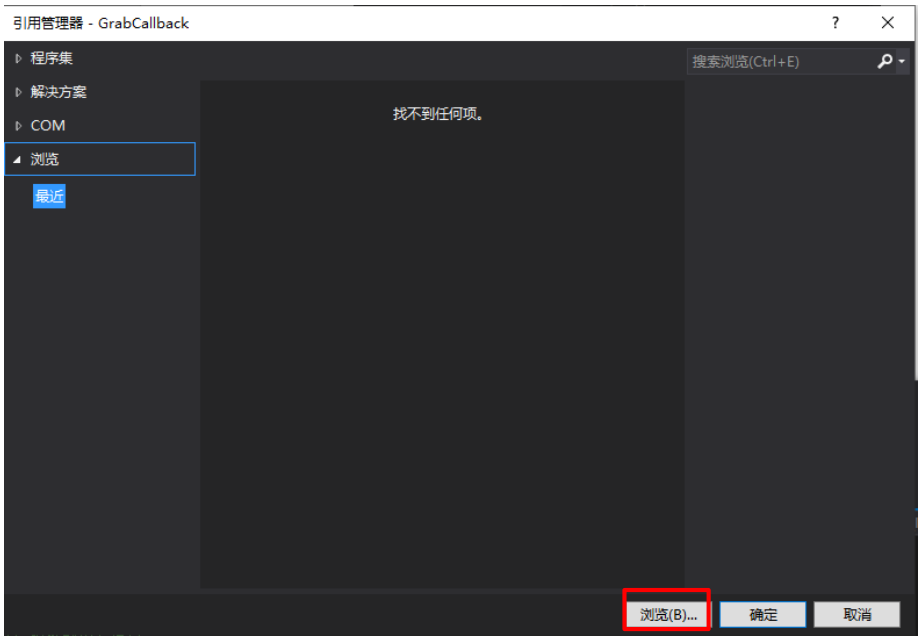
(1) 拷贝 SDK 相关资源文件

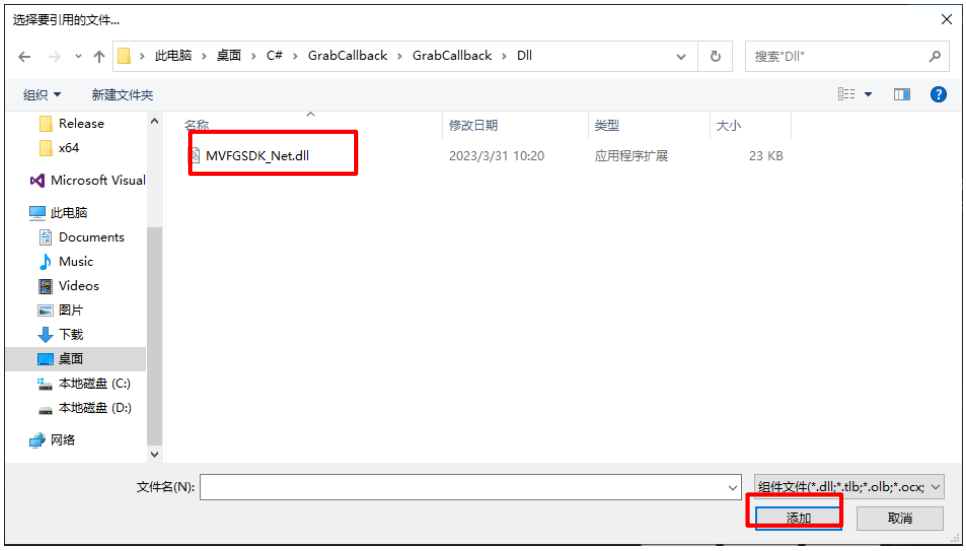


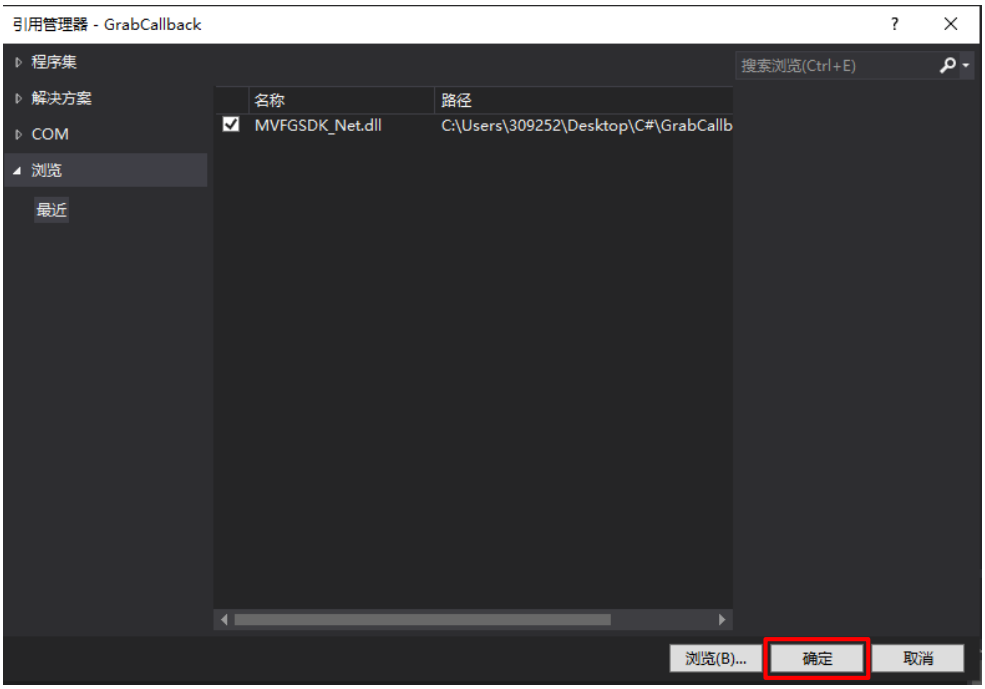
(2) 配置工程

1、项目->添加引用，添加对 MVFGSDK_Net.dll 的引用；





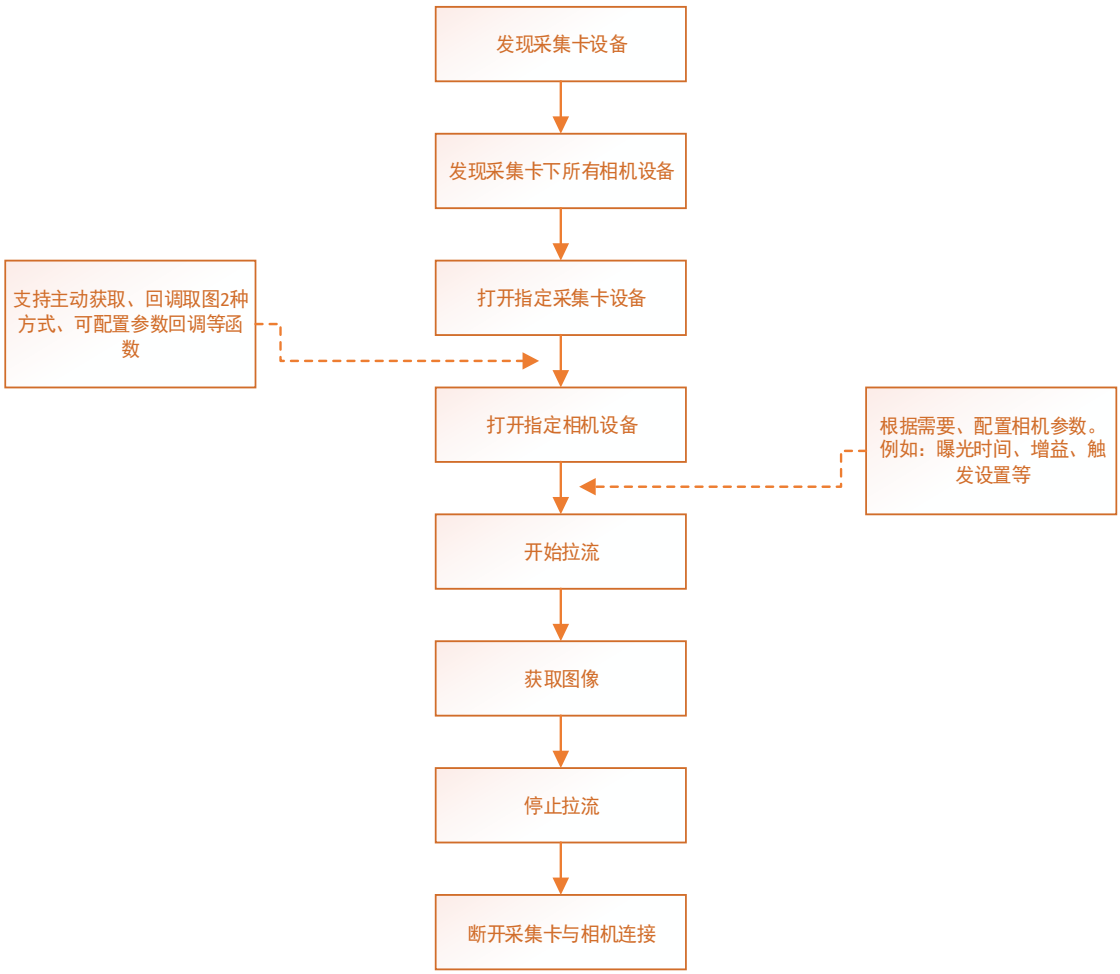




2、在调用代码中添加：using CaptureCard_Net;

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Runtime.InteropServices;
5   using System.Text;
6   using System.Threading;
7   using System.Threading.Tasks;
8   using CaptureCard_Net;
```

2. SDK 的整体调用流程



3. SDK 的各功能接口调用说明

3.1. 系统操作

3.1.1. 枚举采集卡（发现设备）

接口类 ：CardDev

接口 ：public static int IMV_FG_EnumInterface(uint interfaceType,
ref IMVFGDefine.IMV_FG_INTERFACE_INFO_LIST pInterfaceList);

功能说明：枚举采集卡设备。
pInterfaceList [OUT] 设备列表
interfaceType [IN] 待枚举的接口类型，类型可任意组合,如 typeGigEInterface | typeU3vInterface
成功，返回IMV_FG_OK，错误，返回错误码。
1、当interfaceType = typeGigEInterface时，枚举所有GigE网口下的在线设备
2、当interfaceType = typeU3vInterface时，枚举所有USB接口下的在线设备
3、当interfaceType = typeCLInterface时，枚举所有CameraLink接口下的在线设备
4、当interfaceType = typeCXPInterface时，枚举所有CXP接口下的在线设备
5、该接口下的interfaceType支持任意接口类型的组合,如，若枚举所有GigE网口和USB3接口下的在线设备时，可将interfaceType设置为 interfaceType = typeGigEInterface | typeU3vInterface,其它接口类型组合以此类推

参考例程：当前所有例程均有涉及

3.1.2. 枚举相机设备（发现相机设备）

接口类 ：CamDev

接口 ：public static int IMV_FG_EnumDevice(uint nInterfaceType,
ref IMVFGDefine.IMV_FG_DEVICE_INFO_LIST pDeviceList);

功能说明：枚举设备。
nInterfaceType [IN] 枚举设备接口类型
pDeviceList [OUT] 设备列表
枚举采集卡相机设备

参考例程：当前所有例程均有涉及

3.1.3. 获取版本信息

接口类 ：CardDev
接口 ：public static string IMV_FG_GetVersion();
功能说明：获取版本信息
成功时返回版本信息，失败时返回NULL
参考例程：CommPropAccess

3.2. 采集卡操作

3.2.1. 采集卡的连接与关闭

接口类 ：CardDev
接口 ：public int IMV_FG_OpenInterface(uint nIndex);
功能说明：打开采集卡设备
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：当前所有例程均有涉及

接口类 ：CardDev
接口 ：public int IMV_FG_IsOpenInterface();
功能说明：判断采集卡设备是否已打开
打开状态，返回true；关闭状态或者掉线状态，返回false
参考例程：当前所有例程均有涉及

接口类 ：CardDev
接口 ：public int IMV_FG_CloseInterface();
功能说明：关闭采集卡设备
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：当前所有例程均有涉及

3.2.2. 注册事件回调

接口类 ：CardDev
接口 ：public int IMV_FG_SubscribeMsgChannelArg(
IMVFGDefine.IMV_FG_MsgChannelCallBack proc, IntPtr pUser);
功能说明：消息通道事件回调函数注册
proc [IN] 设备消息通道事件回调函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_FG_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：MsgEvents

3.3. 相机操作

3.3.1. 相机的连接与关闭

```
接口类  ：CamDev
接口    ：public int IMV_FG_OpenDevice(IMVFGDefine.IMV_FG_ECreateHandleMode mode,
          int cameraIndex = 0,  string cameraStr = "");
功能说明：打开设备
          mode [IN] 创建句柄模式
          cameraIndex [IN] 打开相机标识
          cameraStr [IN] 打开相机字符串
          成功，返回IMV_FG_OK；错误，返回错误码
参考例程：当前所有例程均有涉及
```

```
接口类  ：CamDev
接口    ：public int IMV_FG_OpenDeviceEx(IMVFGDefine.IMV_FG_ECreateHandleMode mode,
          int cameraIndex = 0,  string cameraStr = "");
功能说明：打开设备(同步采集卡与相机参数)
          mode [IN] 创建句柄模式
          cameraIndex [IN] 打开相机标识
          cameraStr [IN] 打开相机字符串
          成功，返回IMV_FG_OK；错误，返回错误码
```

```
接口类  ：CamDev
接口    ：public int IMV_FG_IsDeviceOpen();
功能说明：判断设备是否已打开
          打开状态，返回true；关闭状态或者掉线状态，返回false
```

```
接口类  ：CamDev
接口    ：public int IMV_FG_CloseDevice();
功能说明：关闭设备
          成功，返回IMV_FG_OK；错误，返回错误码
参考例程：当前所有例程均有涉及
```

3.3.2. 注册事件回调

```
接口类  ：CamDev
接口    ：public int IMV_FG_SubscribeConnectArg(IMVFGDefine.IMV_FG_ConnectCallBack
          proc, IntPtr pUser);
功能说明：相机设备连接状态事件回调注册
```

proc [IN] 设备连接状态事件回调函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_FG_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册

3.4. 相机操作

3.4.1. 流数据设置

接口类 ：CardDev
接口 ：public int IMV_FG_SetBufferCount(uint nSize);
功能说明：设置帧数据缓存个数
handle [IN] 设备句柄
nSize [IN] 缓存数量
成功，返回 IMV_FG_OK；错误，返回错误码
不能在拉流过程中设置

3.4.2. 图像采集

接口类 ：CardDev
接口 ：public int IMV_FG_StartGrabbing();
功能说明：开始取流
成功，返回 IMV_FG_OK；错误，返回错误码
参考例程：Grab

接口类 ：CardDev
接口 ：public int IMV_FG_StartGrabbingEx(UInt64 maxImagesGrabbed,
IMVFGDefine.IMV_FG_EGrabStrategy strategy);
功能说明：开始取流
maxImagesGrabbed [IN] 允许最多的取帧数，达到指定取帧数后停止取流，如果为0，表示忽略此参数连续取流(IMV_FG_StartGrabbing默认0)
strategy [IN] 取流策略,(IMV_FG_StartGrabbing默认使用grabStrartegySequential策略取流)
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：GrabStrategy

接口类 ：CardDev
接口 ：public int IMV_FG_IsGrabbing();
功能说明：判断设备是否正在取流
正在取流，返回true；不在取流，返回false

接口类 ：CardDev
接口 ：public int IMV_FG_StopGrabbing();
功能说明：停止取流
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：Grab

3.4.3. 获取图像数据

接口类 ：CardDev
接口 ：public int IMV_FG_AttachGrabbing(IMVFGDefine.IMV_FG_FrameCallBack proc, IntPtr pUser);
功能说明： 注册帧数据回调函数(异步获取帧数据机制)
proc [IN] 帧数据信息回调函数，建议不要在该函数中处理耗时的操作，否则会阻塞后续帧数据的实时性
pUser[IN] 用户自定义数据，可设为NULL
成功，返回IMV_FG_OK；错误，返回错误码
该异步获取帧数据机制和同步获取帧数据机制(IMV_FG_GetFrame)互斥，对于同一设备，系统中两者只能选其一
只支持一个回调函数，且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：FrameClone、FrameTriggerCount、GrabCallback

接口类 ： CardDev
接口 ： public int IMV_FG_GetFrame(ref IMVFGDefine.IMV_FG_Frame pFrame, uint timeoutMS);
功能说明： 获取一帧图像(同步获取帧数据机制)
pFrame [OUT] 帧数据信息
timeoutMS [IN] 获取一帧图像的超时时间,INFINITE时表示无限等待,直到收到一帧数据或者停止取流。单位是毫秒
成功，返回IMV_FG_OK；错误，返回错误码
该接口不支持多线程调用。
该同步获取帧数据机制和异步获取帧数据机制(IMV_FG_AttachGrabbing)互斥
对于同一设备，系统中两者只能选其一
使用内部缓存获取图像，需要IMV_FG_ReleaseFrame进行释放图像缓存。
参考例程：Grab

接口类 ：CardDev
接口 ：public int IMV_FG_ReleaseFrame(ref IMVFGDefine.IMV_FG_Frame Pframe);
功能说明：释放图像缓存
pFrame [IN] 帧数据信息
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：Grab

3.4.4. 流与图像数据的其他操作

接口类 ：CardDev
接口 ：public int IMV_FG_CloneFrame(ref IMVFGDefine.IMV_FG_Frame pFrame,
ref IMVFGDefine.IMV_FG_Frame pCloneFrame);
功能说明：帧数据深拷贝克隆
pFrame [IN] 克隆源帧数据信息
pCloneFrame [OUT] 新的帧数据信息
成功，返回IMV_FG_OK；错误，返回错误码
使用IMV_FG_ReleaseFrame进行释放图像缓存。
参考例程：FrameClone

接口类 ：CardDev
接口 ：public int IMV_FG_GetStatisticsInfo(
ref IMVFGDefine.IMV_FG_StreamStatisticsInfo pStreamStatsInfo);
功能说明：获取流统计信息(IMV_FG_StartGrabbing / IMV_FG_StartGrabbingEx执行后调用)
pStreamStatsInfo [OUT] 流统计信息数据
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：StreamStatistics

接口类 ：CardDev
接口 ：public int IMV_FG_ResetStatisticsInfo();
功能说明：重置流统计信息(IMV_FG_StartGrabbing / IMV_FG_StartGrabbingEx执行后调用)
成功，返回IMV_FG_OK；错误，返回错误码
参考例程：StreamStatistics

3.5. 属性操作

3.5.1. 采集卡与相机配置的下载、保存及加载配置

接口类 ：CardDev、CamDev
接口 ：public int IMV_FG_DownLoadGenICamXML(string pFullFileName);
功能说明：下载设备描述 XML 文件，并保存到指定路径，如：D:\\xml.zip
pFullFileName [IN] 文件要保存的路径
成功，返回 IMV_FG_OK；错误，返回错误码

接口类 ：CardDev、CamDev

接口　：public int IMV_FG_SaveDeviceCfg(string pFullFileName);

功能说明：保存设备配置到指定的位置。同名文件已存在时，覆盖。

pFullFileName [IN] 导出的设备配置文件全名(含路径)，如：D:\\config.xml 或 D:\\config.mvcfg

成功，返回 IMV_FG_OK；错误，返回错误码

参考例程：LoadAndSaveDeviceCfg

接口类　：CardDev、CamDev

接口　：public int IMV_FG_LoadDeviceCfg(string pFullFileName,

ref IMVFGDefine.IMV_FG_ErrorList pErrorList);

功能说明：从文件加载设备 xml 配置

pFullFileName [IN] 设备配置(xml)文件全名(含路径)，如：D:\\config.xml 或 D:\\config.mvcfg

pErrorList [OUT] 加载失败的属性名列表。存放加载失败的属性上限为 IMV_FG_MAX_ERROR_LIST_NUM。成功，返回IMV_FG_OK；错误，返回错误码

成功，返回 IMV_FG_OK；错误，返回错误码

参考例程：LoadAndSaveDeviceCfg

3.5.2. 属性的判断、获取、设置

① 属性的判断

接口类　：CardDev、CamDev

接口　：public int IMV_FG_FeatureIsAvailable(string pFeatureName);

功能说明：判断属性是否可用

pFeatureName [IN] 属性名

可，返回 true；不可，返回 false

接口类　：CardDev、CamDev

接口　：public int IMV_FG_FeatureIsReadable(string pFeatureName);

功能说明：判断属性是否可读

pFeatureName [IN] 属性名

可，返回 true；不可，返回 false

接口类　：CardDev、CamDev

接口　：public int IMV_FG_FeatureIsWriteable(string pFeatureName);

功能说明：判断属性是否可写

pFeatureName [IN] 属性名

可，返回 true；不可，返回 false

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_FeatureIsStreamable(string pFeatureName);

功能说明：判断属性是否可流

pFeatureName [IN] 属性名

可，返回 true；不可，返回 false

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_FeatureIsValid(string pFeatureName);

功能说明：判断属性是否有效

pFeatureName [IN] 属性名

可，返回 true；不可，返回 false

参考例程：Record

② 属性的获取

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetIntFeatureValue(string pFeatureName, ref Int64 pIntValue);

功能说明：获取整型属性值

pFeatureName [IN] 属性名

pIntValue [OUT] 整型属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetIntFeatureMin(string pFeatureName, ref Int64 pIntValue);

功能说明：获取整型属性可设的最小值

pFeatureName [IN] 属性名

pIntValue [OUT] 整型属性可设的最小值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetIntFeatureMax(string pFeatureName, ref Int64 pIntValue);

功能说明：获取整型属性可设的最大值

pFeatureName [IN] 属性名

pIntValue [OUT] 整型属性可设的最大值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetIntFeatureInc(string pFeatureName, ref Int64 pIntValue);

功能说明：获取整型属性步长

pFeatureName [IN] 属性名

pIntValue [OUT] 整型属性步长

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetDoubleFeatureValue(string pFeatureName, ref double pDoubleValue);

功能说明：获取浮点属性值

pFeatureName [IN] 属性名

pDoubleValue [OUT] 浮点属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetDoubleFeatureMin(string pFeatureName, ref double pDoubleValue);

功能说明：获取浮点属性可设的最小值

pFeatureName [IN] 属性名

pDoubleValue [OUT] 浮点属性可设的最小值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetDoubleFeatureMax(string pFeatureName, ref double pDoubleValue);

功能说明：获取浮点属性可设的最大值

pFeatureName [IN] 属性名

pDoubleValue [OUT] 浮点属性可设的最大值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetBoolFeatureValue(string pFeatureName, ref bool pBoolValue);

功能说明：获取布尔属性值

pFeatureName [IN] 属性名

pBoolValue [OUT] 布尔属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetStringFeatureValue(string pFeatureName, ref IMVFGDefine.IMV_FG_String pStringValue);

功能说明：获取字符串属性值

pFeatureName [IN] 属性名

pStringValue [OUT] 字符串属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetEnumFeatureValue(string pFeatureName, ref UInt64 pEnumValue);

功能说明：获取字符串属性值

pFeatureName [IN] 属性名

pEnumValue [OUT] 字符串属性值

成功，返回IMV_FG_OK；错误，返回错误码

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_GetEnumFeatureSymbol(string pFeatureName, ref IMVFGDefine.IMV_FG_String pEnumSymbol);

功能说明：获取枚举属性 symbol 值

pFeatureName [IN] 属性名

pEnumSymbol [OUT] 枚举属性symbol值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口　：public int IMV_FG_GetEnumFeatureEntryNum(string pFeatureName, ref uint pEntryNum);

功能说明：获取枚举属性可设置个数

pFeatureName [IN] 属性名

pEnumSymbol [OUT] 枚举属性可设置个数

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类　：CardDev、CamDev

接口　：public int IMV_FG_GetEnumFeatureEntrys(string pFeatureName, ref IMVFGDefine.IMV_FG_EnumEntryList pEnumEntryList);

功能说明：获取枚举属性可设置枚举值列表

pFeatureName [IN] 属性名

pEnumSymbol [OUT] 枚举属性可设置枚举值列表

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

③ 属性的设置

接口类　：CardDev、CamDev

接口　：public int IMV_FG_SetIntFeatureValue(string pFeatureName, Int64 pIntValue);

功能说明：设置整型属性值

pFeatureName [IN] 属性名

pIntValue [IN] 待设置的整型属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类　：CardDev、CamDev

接口　：public int IMV_FG_SetDoubleFeatureValue(string pFeatureName, double pDoubleValue);

功能说明：设置浮点属性值

pFeatureName [IN] 属性名

pDoubleValue [IN] 待设置的浮点属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_SetBoolFeatureValue(string pFeatureName, bool pBoolValue);

功能说明：设置布尔属性值

pFeatureName [IN] 属性名

pBoolValue [IN] 待设置的布尔属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_SetEnumFeatureValue(string pFeatureName, UInt64 pEnumValue);

功能说明：设置枚举属性值

pFeatureName [IN] 属性名

pEnumValue [IN] 待设置的枚举属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_SetEnumFeatureSymbol(string pFeatureName,string pEnumSymbol);

功能说明：设置枚举属性 symbol 值

pFeatureName [IN] 属性名

pEnumSymbol [IN] 待设置的枚举属性symbol值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

接口类 ：CardDev、CamDev

接口 ：public int IMV_FG_SetStringFeatureValue(string pFeatureName,string pStrValue);

功能说明：设置字符串属性值

pFeatureName [IN] 属性名

pStrValue [IN] 待设置的字符串属性值

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：CommPropAccess

3.5.3. 命令属性的执行

接口类 ：CardDev、CamDev

接口　：public int IMV_FG_ExecuteCommandFeature(string pFeatureName);

功能说明：执行命令属性

pFeatureName [IN] 属性名

成功，返回IMV_FG_OK；错误，返回错误码

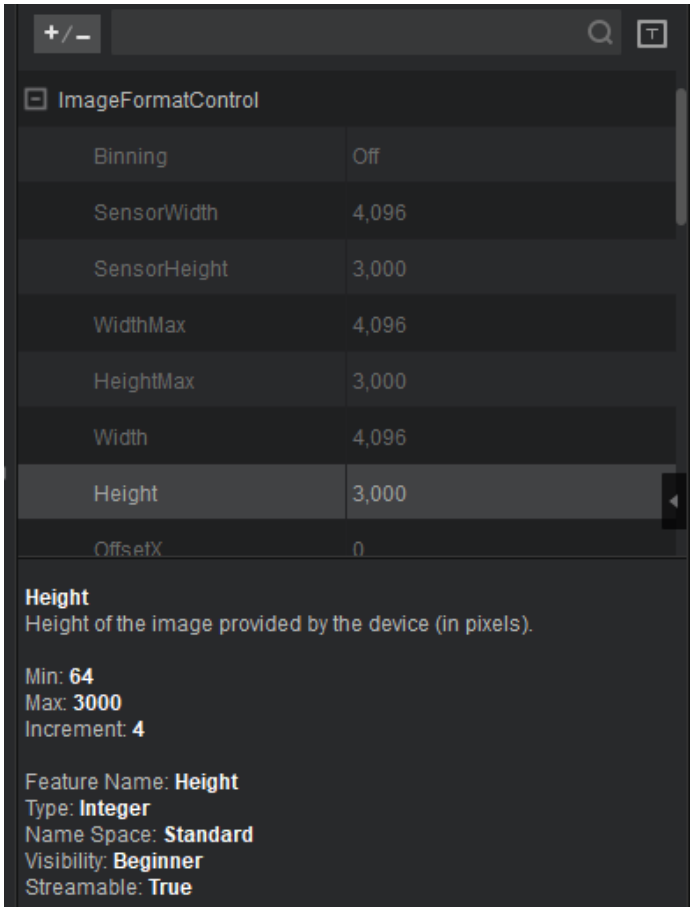
参考例程：CommPropAccess

3.5.4. 属性的查询与使用

打开相机客户端可查询对应的属性，并且可以了解属性的详细信息，如：数据类型、范围等。

1) 整型属性

例：若需获取相机的像素高度信息，找到 Height 属性并选中，下方会显示该属性的数据类型，根据 Type 的显示而定，下图 Height 属性的数据类型为 int 整型。



随后对所找到属性进行获取及修改。

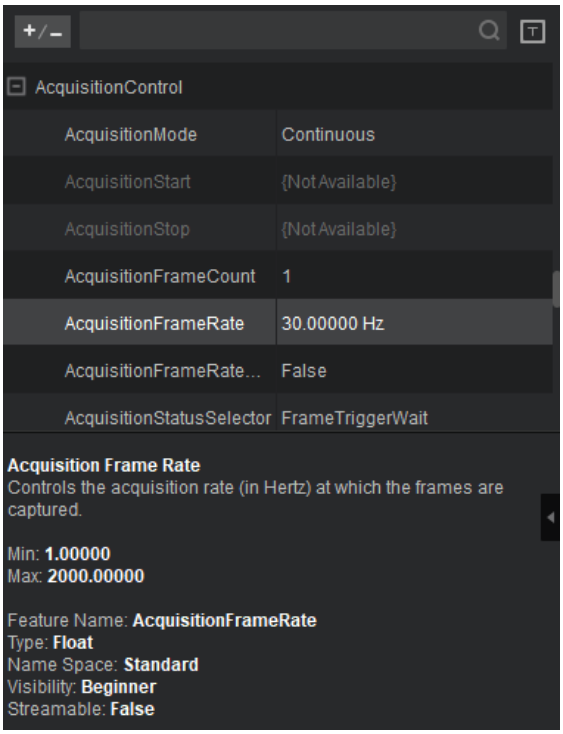
```
int ret = IMVFGDfine.IMV_FG_OK;
Int64 heightValue = 0;
Int64 heightMinValue = 0;
Int64 heightMaxValue = 0;
Int64 incrementValue = 0;
CamDev cam=new CamDev();
```

```
CardDev card=new CardDev();

 // 获取属性值
  ret = cam.IMV_FG_GetIntFeatureValue("Height",ref heightValue);
 if(ret != IMVFGDefine.IMV_FG_OK)
 {
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
 }

 // 获取属性可设的最小值，若无最小值则此操作错误
  ret = cam.IMV_FG_GetIntFeatureMin("Height", ref heightMinValue);
 if(ret != IMVFGDefine.IMV_FG_OK)
 {
    Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
    return ret;
 }
 // 获取属性可设的最大值，若无最大值则此操作错误
  ret = cam.IMV_FG_GetIntFeatureMax("Height", ref heightMaxValue);
 if(ret != IMVFGDefine.IMV_FG_OK)
 {
    Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
    return ret;
 }
 // 获取属性步长，若无步长值则此操作错误
  ret = cam.IMV_FG_GetIntFeatureInc("Height", ref incrementValue);
 if(ret != IMVFGDefine.IMV_FG_OK)
 {
    Console.WriteLine($"Get feature increment value failed! ErrorCode[{ret}]");
    return ret;
 }
 // 设置属性值
  ret = cam.IMV_FG_SetIntFeatureValue("Height", heightValue);
 if(ret != IMVFGDefine.IMV_FG_OK)
 {
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
 }
```

2) 浮点型属性

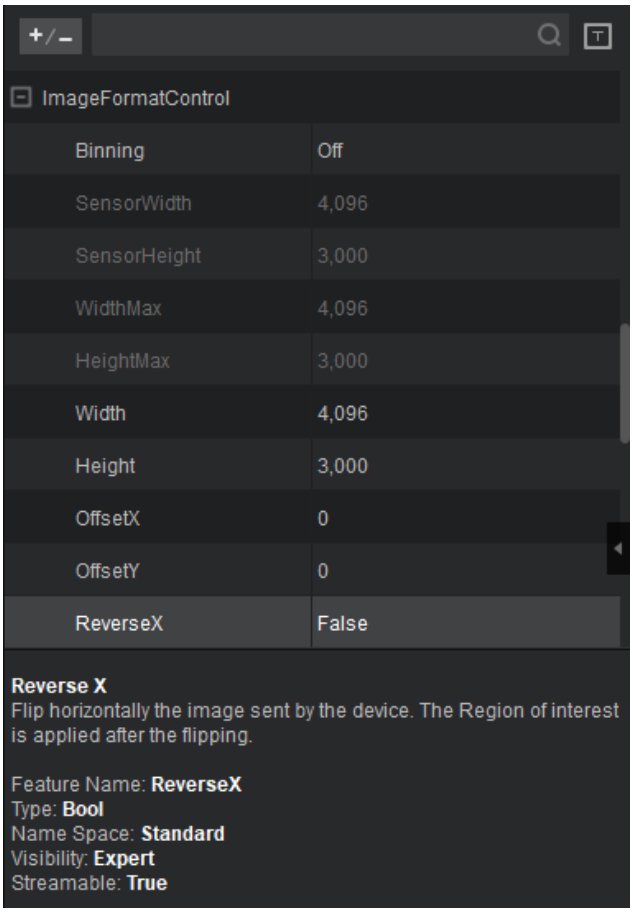



```
int ret = IMVFGDfine.IMV_FG_OK;
double framerate = 0;
double frameratemin = 0;
double frameratemax = 0;

// 获取属性值
ret = cam.IMV_FG_GetDoubleFeatureValue ("AcquisitionFrameRate", ref framerate);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 获取属性可设的最小值，若无最小值则此操作错误
ret = cam.IMV_FG_GetDoubleFeatureMin("AcquisitionFrameRate", ref frameratemin);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
    return ret;
}
// 获取属性可设的最大值，若无最大值则此操作错误
ret = cam.IMV_FG_GetDoubleFeatureMax("AcquisitionFrameRate", ref frameratemax);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
    return ret;
}
```

```
// 设置属性值
ret = cam.IMV_FG_SetDoubleFeatureValue("AcquisitionFrameRate", framerate);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

3) 布尔属性

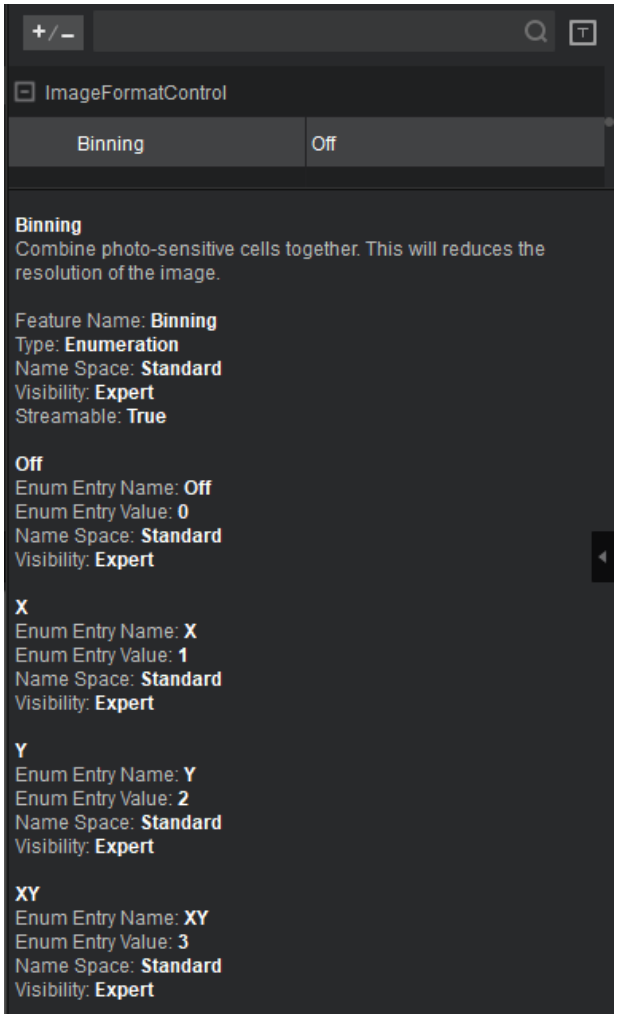



```
int ret = IMVFGDfine.IMV_FG_OK;
bool reversevalue = 0;

// 获取属性值
ret = cam.IMV_FG_GetBoolFeatureValue ("ReverseX", ref reversevalue);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 设置属性值
ret = cam.IMV_FG_SetBoolFeatureValue("ReverseX", reversevalue);
if(ret != IMVFGDefine.IMV_FG_OK)
```

```
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

4) 枚举属性

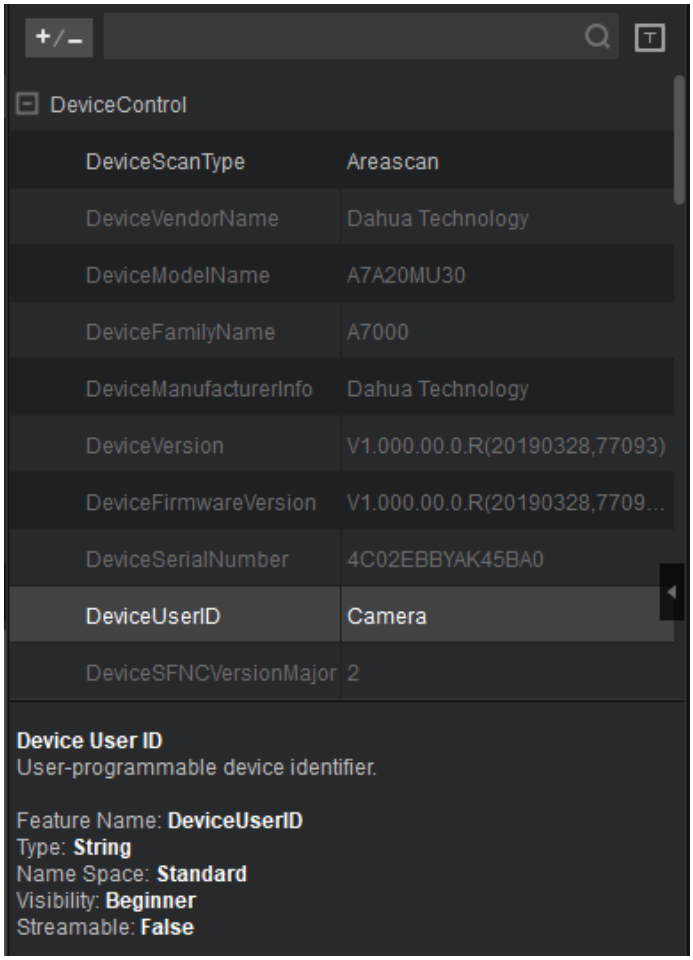



```
int ret = IMVFGDfine.IMV_FG_OK;
UInt64 enumValue = 0;
IMVFGDefine.IMV_FG_String pEnumSymbol = new IMVFGDefine.IMV_FG_String();
uint pEntryNum = 0;
IMVFGDefine.IMV_FG_EnumEntryList pEnumEntryList = new
IMVFGDefine.IMV_FG_EnumEntryList();

// 获取属性值的可设枚举值列表
ret = cam.IMV_FG_GetEnumFeatureEntrys("Binning", ref pEnumEntryList);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
```

```
}
// 获取属性可设枚举值的个数
ret = cam.IMV_FG_GetEnumFeatureEntryNum("Binning", ref pEntryNum);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 获取属性symbol值
ret = cam.IMV_FG_GetEnumFeatureSymbol("Binning", ref pEnumSymbol);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 获取属性值
ret = cam.IMV_FG_GetEnumFeatureValue ("Binning", ref enumValue);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 设置属性symbol值
ret = cam.IMV_FG_SetEnumFeatureSymbol("Binning", pEnumSymbol);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 设置属性值
ret = cam.IMV_FG_SetEnumFeatureValue("Binning", enumValue);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

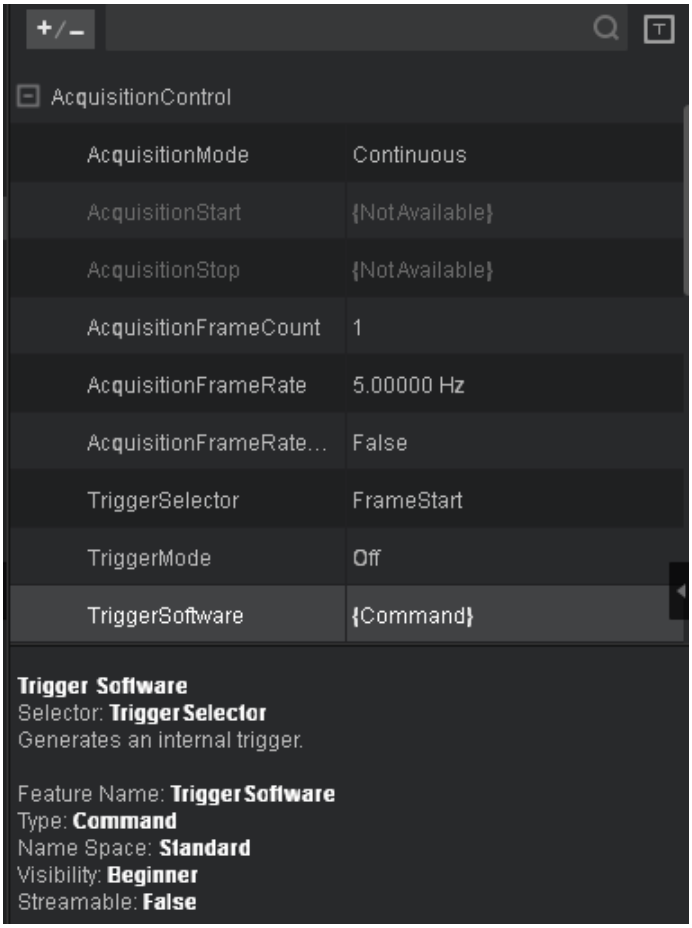
5) 字符串属性




```
int ret = IMVFGDfine.IMV_FG_OK;
string id = "";

// 获取属性值
ret = cam.IMV_FG_GetStringFeatureValue ("DeviceUserID", ref id);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 设置属性值
ret = cam.IMV_FG_SetStringFeatureValue("DeviceUserID", id);
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

6） 命令属性




```
int ret = IMVFGDfine.IMV_FG_OK;
// 执行命令属性值
ret = cam.IMV_FG_ExecuteCommandFeature ("TriggerSoftware");
if(ret != IMVFGDefine.IMV_FG_OK)
{
    Console.WriteLine($"Execute Command feature failed! ErrorCode[{ret}]");
    return ret;
}
```

3.6. 图像操作

3.6.1.录像相关

接口类 ：CardDev

接口 ：public int IMV_FG_OpenRecord(ref IMVFGDefine.IMV_FG_RecordParam pstRecordParam);

功能说明：打开录像

pstRecordParam [IN] 录像参数结构体

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：Record

接口类 ：CardDev

接口 ：public int IMV_FG_InputOneFrame(ref IMVFGDefine.IMV_FG_RecordFrameInfoParam pstRecordFrameInfoParam);

功能说明：录制一帧图像

pstRecordFrameInfoParam [IN] 录像用帧信息结构体

成功，返回IMV_FG_OK；错误，返回错误码

参考例程：Record

3.6.2. 图像相关

接口类 ：CardDev

接口 ：public int IMV_FG_PixelConvert(ref IMVFGDefine.IMV_FG_PixelConvertParam pstPixelConvertParam);

功能说明：像素格式转换

pstPixelConvertParam [IN][OUT] 像素格式转换参数结构体

成功，返回IMV_FG_OK；错误，返回错误码

只支持转化成目标像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8 / gvspPixelBGRA8

通过该接口将原始图像数据转换成用户所需的像素格式并存放在调用者指定内存中。

像素格式为YUV411Packed的时，图像宽须能被4整除

像素格式为YUV422Packed的时，图像宽须能被2整除

像素格式为YUYVPacked的时，图像宽须能被2整除

转换后的图像:数据存储是从最上面第一行开始的，这个是相机数据的默认存储方向

参考例程：SoftTrigger、Grab、GrabCallback

接口类 ：CardDev

接口 ：public int IMV_FG_FlipImage(ref IMVFGDefine.IMV_FG_FlipImageParam pstFlipImageParam);

功能说明：图像翻转

pstFlipImageParam [IN][OUT] 图像翻转参数结构体

成功，返回IMV_FG_OK；错误，返回错误码

只支持像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8的图像的垂直和水平翻转。

通过该接口将原始图像数据翻转后并存放在调用者指定内存中。

参考例程：ImageFlip

接口类 ：CardDev

接口 ：public int IMV_FG_RotateImage(ref IMVFGDefine.IMV_FG_RotateImageParam pstRotateImageParam);

功能说明：图像顺时针旋转

pstRotateImageParam [IN][OUT] 图像旋转参数结构体

成功，返回IMV_FG_OK；错误，返回错误码

只支持gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8格式数据的90/180/270度顺时针旋转。

通过该接口将原始图像数据旋转后并存放在调用者指定内存中。

参考例程：ImageRotate

接口类 ：CardDev

接口 ：public int IMV_FG_SaveImageToFile(ref IMVFGDefine.IMV_FG_SaveImageToFileParam pstSaveFileParam);

功能说明：保存图像到文件

pstSaveFileParam [IN] 保存图片到文件参数结构体

成功，返回IMV_FG_OK；错误，返回错误码

该接口支持保存BMP/JPEG/PNG/TIFF。

JPEG格式最大支持宽高为65500

参考例程：SaveImageToFile

4. 第三方平台的图像对象转换方法

4.1. Halcon 对象（HObject）

4.1.1. 相机图像格式为 Mono8 时

相机图像格式为 Mono8 时，直接用采集到的图像数据构建 Halcon 对象。

此处，假设 pFrame 为 SDK 取流得到的 IMV_FG_Frame 数据帧结构体

```
HObject ho_image;

HOperatorSet.GenImage1Extern(out ho_image,
                             "byte",
                             (int)pFrame.frameInfo.width,
                             (int)pFrame.frameInfo.height,
                             (HTuple)pData,
                             0);
```

4.1.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）

(1) 将采集到的相机图像数据，转换成 BGR24 格式,假设 pData 是转码之后的 BGR 数据

(2) 使用转换后的 BGR24 格式图像数据，构建 Halcon 图像对象（HObject）

```
HObject ho_image;
HOperatorSet.GenImageInterleaved(out ho_image,
                                 (HTuple)pData,
                                 "bgr",
                                 (int)pFrame.frameInfo.width,
                                 (int)pFrame.frameInfo.height,
                                 0,
                                 "byte",
                                 (int)pFrame.frameInfo.width,
                                 (int)pFrame.frameInfo.height,
                                 0, 0, 8, 0);
```

4.2. Mvp 图像对象（MvpImage）

4.2.1. 相机图像格式为 Mono8 时

Mono10、Mono12 等格式需参考 3.6.2 图像格式转换为 Mono8。相机图像格式为 Mono8 时，可直接用采集到的图像数据构建对象。

此处，假设 pFrame 为 SDK 取流得到的 IMV_FG_Frame 数据帧结构体

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImage1(MvpImgDataType.MVP_IMGDTP_U8,
                      (int) pFrame.frameInfo.width,
                      (int)pFrame.frameInfo.height,
                      pData);
```

4.2.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）

将采集到的相机图像数据，转换成 BGR24 格式(参考 Grab 例程 ConvertToBGR24)，假设 pData 是转码之后的 BGR 数据

使用转换后的 BGR24 格式图像数据，构建 Mvp 图像对象

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImageInterleaved(pData,
                                EColorSpace.MVP_COLOR_SPACE_BGR,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                MvpImgDataType.MVP_IMGDTP_U8,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0, 0, -1,0);
```

5. 配套例程说明

例程路径： 【SDK 安装目录】\Development\Samples\C#\IMVFG\

例程名	说明
BasicDemo	基本例程
CommPropAccess	通用属性使用例程
FrameClone	帧克隆
FrameTriggerCount	帧触发统计
Grab	主动取图显示
GrabCallback	回调取图显示
GrabStrategy	拉流策略
ImageConvert	图像转码
ImageFlip	图像翻转
ImageRotate	图像顺时针旋转
LineTrigger	硬件触发取图
MsgEvents	消息通道
LoadAndSaveDeviceCfg	保存和加载相机属性配置文件
SaveImageToBmp	保存 BIN 文件和 BMP 图像
SaveImageToFile	保存图像到本地
SoftwareTrigger	软件触发取图
StreamStatistics	流统计

- END -